

Machine Learning

Cybersecurity Project

Part 1: Auditing and Evaluating an Intrusion Detection Dataset

Module leader: Prof Mounir Ghogho
Lab instructor: Dr Abdelghani Ghanem

Overview

In this lab, you will study the introduced by [CICIDS2017](#) network-intrusion dataset. It contains benign network traffic and several attacks collected over five days in a controlled network.

Your goal is not simply to obtain the highest score. You will investigate:

Did the model learn general attack behavior, or did it learn shortcuts specific to the data collection process?

The project focuses on data quality, exploratory analysis, class imbalance, evaluation protocols, leakage, and careful interpretation. You will use simple classifiers so that the main emphasis remains on the data and experiment.

Roadmap

1. understand how the dataset was created;
2. audit its quality and class balance;
3. explore features using hypotheses about network behavior;
4. establish simple classification baselines;
5. compare random, grouped, and chronological evaluation protocols;
6. test whether models rely on shortcuts;
7. translate statistical results into operational consequences.

Dataset and prediction tasks

Download the original `GeneratedLabelledFlows.zip` archive from:

<https://www.unb.ca/cic/datasets/ids-2017.html>

Do not use only `MachineLearningCSV.zip`. The original labelled-flow files contain metadata needed below, including timestamps, addresses, ports, protocol, and source filename. Raw packet captures are not required.

Each row represents one bidirectional network flow. `CICFlowMeter`¹ constructs flows from captured packets and computes features such as duration, packet and byte counts, packet-size statistics, inter-arrival times, TCP flags, and active or idle periods. Its processing rules may also create artifacts that a model can learn.

Represent observation (i) by

$$\mathbf{x}^{(i)} = (x_1^{(i)}, \dots, x_d^{(i)}),$$

with label $y^{(i)}$. You will consider:

- **binary detection:** benign versus attack;
- **multiclass detection:** benign versus individual attack types.

Keep enough metadata to reconstruct the source file, day, and time of every flow, even if these variables are excluded from the behavioral model.

Capture schedule

- Monday: benign traffic only;
- Tuesday: FTP and SSH brute-force attacks;
- Wednesday: DoS attacks and Heartbleed;
- Thursday: web attacks and infiltration;
- Friday: botnet traffic, port scans, and DDoS.

Attack type is associated with the day, time, hosts, service, and attack tool. A model may learn these clues instead of general attack behavior.

State evidence carefully. Later studies reported labeling and packet- capture problems in CICIDS2017. Distinguish reported problems from those you verify in the CSV files. Identical feature vectors with different labels show ambiguity, but do not prove that either label is wrong. Packet-level problems generally cannot be confirmed from flow summaries alone.

Evaluation rules to follow from the beginning

These rules prevent validation or test information from influencing model development.

¹`CICFlowMeter` is a software which converts captured network packets into bidirectional flow records and computes features. In CICIDS2017, these flow summaries form most of the model inputs. Its processing rules and possible errors can also introduce artifacts that a classifier may learn.

Rule 1: Create the protocols first

Before comparing features across labels, create and save the random-row, grouped, and chronological protocols defined in Part 5. For each protocol, save the training, validation, and test row identifiers and record all random seeds. Use validation data for feature choices, hyperparameters, and thresholds. Do not inspect test results until all choices are frozen. Validation must obey the same grouping or time-ordering rule as testing.

Rule 2: Construct one safe exploratory dataset

Parts 2 and 3 use one shared dataset for label-informed plots:

$$\mathcal{D}_{\text{explore}} = \mathcal{D}_{\text{train}}^{\text{random}} \cap \mathcal{D}_{\text{train}}^{\text{grouped}} \cap \mathcal{D}_{\text{train}}^{\text{chronological}}.$$

Assign every row a stable identifier, create the protocols, and retain the identifiers found in all three training partitions. This is not a fourth split. It is the common subset that is safe to inspect under every protocol.

If it is too small or unrepresentative, reserve a separate exploratory subset before inspecting labels. Exclude that subset and its related groups from all validation and test sets. Explain your choice.

Rule 3: Fit preprocessing on training data only

You may check the raw schema, timestamps, missing values, and duplicates before splitting. Afterward, fit imputation, scaling, feature selection, dimensionality reduction, and resampling on the relevant training set only. Apply the fitted operation unchanged to validation and test data. Prevent exact duplicates from crossing partitions and document conflicting labels.

Part 1: Understand how the data were generated

Purpose. Understand what one row represents and how the collection process may create shortcuts.

Using the original paper and official documentation:

1. Create a table connecting each attack to its day, time interval, attacker, victim, service, and tool when available.
2. Identify at least three variables or flow properties that may act as proxies for the label. A proxy predicts the label without necessarily describing the attack mechanism.
3. Explain the difference between detecting an attack mechanism and recognizing a specific tool, host, or capture period.
4. Before plotting, state two hypotheses about expected network behavior.

Report. Include the schedule table, possible proxies, and hypotheses.

Part 2: Audit data quality and class balance

Purpose. Identify problems that could invalidate or distort later experiments.

Use three data scopes:

- **Full raw dataset:** establish facts about schema problems, invalid values, duplicates, and conflicting labels. Do not select predictive features from this global audit.
- $\mathcal{D}_{\text{explore}}$: make label-informed plots without inspecting validation or test observations. These plots can reveal which features separate the classes. They may therefore influence feature selection, preprocessing, and model design.
- **Each protocol's training set:** fit the actual cleaning and preprocessing rules used by that protocol.

Tasks

1. On the full dataset, report rows, columns, source files, and examples per class. Repeat class counts for $(\mathcal{D}_{\text{explore}})$. Plot class frequencies on linear and logarithmic scales.
2. Count missing values, positive and negative infinity, malformed timestamps, and inconsistent column names.
3. Find exact duplicate rows. Repeat after excluding labels and metadata.
4. Find identical predictor vectors assigned different labels.
5. Using $(\mathcal{D}_{\text{explore}})$, inspect feature ranges and distributions by class. Identify strongly correlated or algebraically redundant features.
6. Within each protocol's training set, identify constant and nearly constant features and define their treatment.
7. Produce a table giving each problem, the number of affected rows or columns, the proposed treatment, and its justification.

Required output. Include class-frequency plots, a table of missing and non-finite values, at least one feature-distribution plot by class, a correlation plot, and the data-quality table. Do not silently delete rows.

Part 3: Explore network behavior

Purpose. Determine whether feature differences have a plausible connection to network behavior. Use $(\mathcal{D}_{\text{explore}})$ for all label-informed plots.

Organize features into traffic volume, packet size, timing, directionality, TCP behavior, and activity features.

1. Select at least two attacks that use different mechanisms.

2. For each attack, write a behavioral hypothesis before plotting.
3. Choose suitable plots, such as empirical cumulative distribution functions, boxplots, violin plots, or log-scaled scatter plots.
4. Decide whether the result is consistent with the proposed mechanism. Give at least one alternative explanation linked to the collection scenario.

PCA, t-SNE, or UMAP may be used as optional visualizations. Visible clusters are not proof that a model will generalize to another network.

Report. For each attack, present the hypothesis, plot, observation, and at least one competing explanation.

Part 4: Establish simple classification baselines

Purpose. Create understandable reference results before making more complex claims.

Using the predeclared stratified random split, train for both prediction tasks:

- a majority-class baseline, which always predicts the most frequent training class;
- a decision tree with maximum depth 3;
- logistic regression;
- k-nearest neighbors, abbreviated kNN.

Fit preprocessing on training data only. Scale features for logistic regression and kNN. Use validation data to choose k and logistic-regression regularization. Keep tree depth fixed at 3 and report all tested values.

For binary detection, treat attack as positive and select by attack (F_1). For multiclass detection, select by macro- (F_1) . For one class,

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}, \quad F_1 = 2 \frac{\text{Precision Recall}}{\text{Precision} + \text{Recall}}.$$

Report. Include balanced accuracy, macro- (F_1) , weighted (F_1) , class-wise recall, confusion matrices, and selected hyperparameters. Explain why macro- (F_1) and weighted (F_1) can differ. Interpret the depth-3 tree and state whether its rules reflect behavior or scenario-specific clues.

Part 5: Compare evaluation protocols

Purpose. Show that the split determines the generalization question.

1. **Random-row split:** assign individual flows randomly. This tests another flow from a similar mixture of collection conditions.
2. **Grouped split:** keep related flows together. Choose and justify a group such as a five-tuple, source-destination pair, capture file, or attack episode.

3. **Chronological split:** train on the earliest period, validate on a later period, and test on the latest period. Report cutoffs. Do not use Monday alone for training because it has no attacks. Exclude a flow from an earlier partition if it crosses a time boundary.

For every protocol:

1. state the deployment question it represents;
2. verify grouping or time-ordering constraints;
3. tabulate attack-category counts in all subsets, including zeros;
4. identify test categories absent from training;
5. refit the complete pipeline using the same feature definitions, model families, hyperparameter search, and metrics;
6. quantify and explain changes in performance.

Important limitation: unseen classes

Attack categories are tied to days. Holding out one day can place test classes that never occur in training. A classifier cannot learn an unseen class. Report this open-set problem rather than hiding it.

You may use binary labels or a within-class earlier-versus-later split for the temporal comparison. Binary labels still leave unseen attack mechanisms. A within-class temporal split may not be globally chronological, so name it accurately. Performance can also change because of class proportions, attack coverage, training size, hosts, or services. Do not attribute every decrease to leakage removal.

Part 6: Probe shortcuts and leakage

Purpose. Identify which kinds of information support performance.

Choose the split that best represents deployment before examining its test scores. Use one fixed classifier configuration and the same partitions below. Refit every pipeline separately. Define non-overlapping metadata and behavioral feature lists, then compare:

1. **Metadata-only:** time-derived features, ports, protocol, endpoints, and other identifiers or scenario variables.
2. **Behavior-only:** timing, volume, packet size, directionality, and TCP behavior, excluding identifiers and timestamps.
3. **Reduced redundancy:** retain one representative from each strongly correlated behavioral group, determined on training data.
4. **Label shuffle:** permute training labels for the behavior-only model while keeping validation and test labels unchanged. Refit every label-dependent step and repeat with at least

five predeclared seeds. Compare with an imbalance-aware null baseline, not universal 50% accuracy.

Discuss whether metadata predicts surprisingly well, how much performance remains without direct scenario clues, and which features remain important under random and grouped splits. State the importance method and explain what additional evidence would distinguish an attack mechanism from a shortcut. These probes suggest scenario dependence; they do not prove leakage.

Part 7: Evaluate operational consequences

Purpose. Convert statistical results into quantities that matter when operating an intrusion-detection system.

Use validation data to choose one threshold. Apply it once to test data without changing it.

1. Plot a precision-recall curve.
2. On validation data, examine the trade-off between attack recall and false alarms as the threshold changes.
3. State a target false-positive rate and select a threshold that meets it if possible.
4. On test data, report recall, false-positive rate, and

$$\text{FP}_{100k} = 100000 \frac{FP}{N_{\text{benign}}},$$

the number of false alarms per 100,000 benign flows.

5. Justify the threshold using false-positive and false-negative costs and an assumed deployment attack prevalence.
6. Explain why precision may change when deployment prevalence differs from CICIDS2017.

Accuracy alone is not an acceptable operational metric.

Final discussion

Answer these questions in the report:

1. Which result changed most under the deployment-aligned split?
2. What evidence suggests that a model learned a shortcut?
3. Which conclusions remained stable across preprocessing and splits?
4. What can the best model legitimately claim to detect?
5. What can it not claim to detect?
6. What additional data are needed to test a new organization, host, day, or implementation of the same attack?

Deliverables

Submit:

- well-documented and reproducible Python code;
- a data-quality summary generated by the code;
- all required plots, tables, and confusion matrices;
- a 4 to 6 page report describing methods, results, and limitations;
- a one-paragraph model card stating intended use, evaluation setting, known limitations, and unsupported claims.

Report random seeds and make the split strategy easy to find. Do not try many seeds and report only the most favorable result.

Learning outcome

After completing the project, you should be able to explain why intrusion detection performance depends on how traffic was collected, labeled, represented, cleaned, and split. You should also be able to identify possible leakage, select metrics for imbalanced data, and make claims supported by the evaluation protocol.

References

1. I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, “Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization,” *ICISSP*, pp. 108–116, 2018. <https://doi.org/10.5220/0006639801080116>
2. Canadian Institute for Cybersecurity, University of New Brunswick, “Intrusion Detection Evaluation Dataset (CIC-IDS2017).” <https://www.unb.ca/cic/datasets/ids-2017.html>
3. G. Engelen, V. Rimmer, and W. Joosen, “Troubleshooting an Intrusion Detection Dataset: The CICIDS2017 Case Study,” *2021 IEEE Security and Privacy Workshops*, pp. 7–12, 2021. <https://doi.org/10.1109/SPW53761.2021.00009>
4. M. Lanvin, P.-F. Gimenez, Y. Han, F. Majorczyk, L. Mé, and É. Totel, “Errors in the CICIDS2017 Dataset and the Significant Differences in Detection Performances It Makes,” *Risks and Security of Internet and Systems*, LNCS 13857, pp. 18–33, 2023. https://doi.org/10.1007/978-3-031-31108-6_2